

Securing Language Models Against Vulnerability Encoding

Rui Melo

up202408602@up.pt

University of Porto, Faculty of Engineering

Porto, Portugal

Abstract

The integration of Large Language Models (LLMs) into software development workflows has transformed automated programming but introduced significant security challenges. LLMs often generate vulnerable code due to the insecure patterns present in training data, leading to the generation of code vulnerable to threats such as SQL injection, cross-site scripting, and buffer overflows. Existing mitigation strategies, including static and dynamic analysis tools and prompt engineering, are reactive rather than preventive. Recent advances in model training, such as fine-tuning and adversarial training, offer promising avenues for enhancing the security of LLM-generated code. This paper explores different methodologies and proposes an evaluation framework to embed security directly into AI-assisted programming. By integrating security into model training and assessment, we aim to establish a robust standard for secure AI-driven programming.

CCS Concepts

• Security and privacy → Software security engineering.

Keywords

LLMs, Security Vulnerabilities, AI-Assisted Coding, Secure Code Generation

ACM Reference Format:

Rui Melo. 2025. Securing Language Models Against Vulnerability Encoding. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*, June 23–28, 2025, Trondheim, Norway. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3696630.3731466>

1 Problem Statement

The adoption of LLMs into software development has significantly enhanced productivity, yet security remains a major concern. Trained on vast corpora containing both secure and insecure coding practices, LLMs frequently generate vulnerable code [9, 13]. Empirical studies show that Artificial Intelligence (AI)-generated code is prone

This work was partially funded by projects AISym4Med (101095387) supported by Horizon Europe Cluster 1: Health, ConnectedHealth (n.º 46858), supported by Competitiveness and Internationalisation Operational Programme (POCI) and Lisbon Regional Operational Programme (LISBOA 2020), under the PORTUGAL 2020 Partnership Agreement, through the European Regional Development Fund (ERDF) and Agenda "Center for Responsible AI", nr. C645008882-00000055, investment project nr. 62, financed by the Recovery and Resilience Plan (PRR) and by European Union - NextGeneration EU, and also by FCT plurianual funding for 2020-2023 of LIACC (UIDB/00027/2020 UIDP/00027/2020).



This work is licensed under a Creative Commons Attribution 4.0 International License. FSE Companion '25, Trondheim, Norway

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1276-0/2025/06

<https://doi.org/10.1145/3696630.3731466>

to security weaknesses such as command injection, improper input validation and buffer overflows. For instance, the following C function, generated by an LLM, contains a critical buffer overflow vulnerability:

```
void vulnerable_function() {
    char buffer[20];
    printf("Enter text: ");
    scanf("%s", buffer);
    printf("You entered: %s\n", buffer);
}
```

In this function, the use of `scanf("%s", buffer);` without specifying a maximum field width allows an attacker to input a string longer than the allocated buffer size of 20 bytes. This can overwrite adjacent memory, causing unpredictable behaviour such as program crashes or arbitrary code execution. Existing mitigation methods primarily rely on reactive approaches, such as static and dynamic analysis [3], or prompt engineering [12]. However, these do not address the root cause: the insecure generation process itself.

Recent research suggests that integrating security mechanisms into LLM training, such as reinforcement learning with security-aware rewards [7], fine-tuning on vulnerability-free datasets [5, 14], and contrastive learning to distinguish secure from insecure code [15]. Additionally, Explainable AI (XAI) techniques, such as steering vectors [1, 11], can improve model interpretability and guide it toward safer coding patterns.

2 Background

AI-driven code generation has sped up development but raised security and reliability concerns. LLMs trained on public code may replicate insecure patterns, outdated constructs, and weak validation, leading to vulnerabilities.

A study on GitHub Copilot-generated code found that 29.5% of Python and 24.2% of JavaScript snippets contained security flaws spanning 43 different Common Weakness Enumerations (CWE), including CWE-78 (OS Command Injection) and CWE-94 (Improper Control of Code Generation) [4]. These findings highlight the urgent need for security-aware training methodologies.

Ensuring the security of generated code remains a challenge due to inherent limitations, such as the lack of contextual awareness in LLMs and the potential use of imperfect training data [2]. Static and dynamic analysis tools play a crucial role in detecting security vulnerabilities. While static analysis identifies potential issues pre-execution, dynamic analysis captures runtime anomalies. Nevertheless, both approaches have limitations, necessitating continuous research into hybrid and adversarial techniques to improve security assurance. Understanding and addressing these challenges is essential for developing AI-driven code generation frameworks that align with modern secure coding standards.

3 Related Work

To mitigate security vulnerabilities in AI-generated code, we propose a security-enhanced LLM training framework that integrates:

Fine-Tuning on Security-Focused Datasets: Augmenting training data with both secure and insecure code collected from real-world repositories, open-source security patch datasets, and vulnerability databases to help LLMs distinguish safe patterns. [10]

Contrastive Learning for Secure Code Prioritization: Training LLMs to differentiate between secure and insecure coding practices through contrastive loss functions, leveraging curated datasets with labelled security vulnerabilities and best practices [8]. These datasets will include real-world insecure code snippets, manually annotated secure alternatives, and adversarially generated samples to enhance robustness. Additionally, we will proceed with fine-tuning to enable models to distinguish semantic differences between secure and insecure code structures, enhancing their ability to generalize to unseen vulnerabilities.

Reinforcement Learning with Security-Aware Rewards: Penalizing vulnerabilities and incentivising security best practices using automated static and dynamic analysis feedback [6]. Static analysis will be leveraged to detect insecure code patterns before execution, identifying potential issues such as buffer overflows and improper input validation. Dynamic analysis will complement this by monitoring runtime behaviours and capturing execution anomalies that could indicate security flaws. By combining these two approaches, the training process can provide more comprehensive security-aware rewards, ensuring models learn to generate syntactically correct code and resist exploitation.

Explainable AI for Security Interpretability: Employing Sparse AutoEncoders (SAEs) and steering vectors to improve transparency and control over LLM decision-making, allowing the model to prioritize secure coding practices. SAEs help identify hidden patterns in code representations while steering vectors adjust the model's latent space to emphasize security-relevant features. This approach ensures that the model not only generates more secure code but also provides interpretability, helping developers understand and validate the security of AI-generated code.

Comprehensive Security Evaluation: We aim to build a rigorous evaluation framework to assess the security of LLMs-generated code through the incorporation of benchmark datasets featuring diverse vulnerability scenarios and leveraging automated security tools to assess security robustness.

These methodologies aim to shift AI-assisted coding from reactive fixes to proactive vulnerability prevention, making security an intrinsic property of generated code rather than an afterthought.

4 Conclusion

Ensuring the security of AI-generated code requires a shift from reactive detection to proactive security-aware model training. Security-enriched fine-tuning, contrastive learning, reinforcement learning, and XAI-driven interpretability offer valuable avenues for mitigating vulnerabilities in LLMs, enhancing their robustness and reliability. By embedding security principles directly into model training and evaluation, this research contributes to making AI-assisted programming more secure and reliable.

Future work will refine contrastive learning for vulnerability detection, optimize security-aware rewards, and expand XAI for better interpretability. Establishing standardized benchmarks will be key to evaluating and improving secure LLMs. By embedding security into the model's core, we promote a secure-by-design approach that boosts AI-driven development robustness and functionality.

References

- [1] Sanjeev Arora, Yuanzhi Li, Yingyu Liang, Tengyu Ma, and Andrej Risteski. 2018. Linear Algebraic Structure of Word Senses, with Applications to Polysemy. *Transactions of the Association for Computational Linguistics* 6 (2018), 483–495. doi:10.1162/tacl_a_00034
- [2] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, Zhen Ming, and Jiang. 2023. GitHub Copilot AI pair programmer: Asset or Liability? arXiv:2206.15331 [cs.SE] <https://arxiv.org/abs/2206.15331>
- [3] Greta Dolcetti, Vincenzo Arceri, Eleonora Iotti, Sergio Maffei, Agostino Cortesi, and Enea Zaffanella. 2025. Helping LLMs Improve Code Generation Using Feedback from Testing and Static Analysis. arXiv:2412.14841 [cs.SE] <https://arxiv.org/abs/2412.14841>
- [4] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. 2024. Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. arXiv:2310.02059 [cs.SE] <https://arxiv.org/abs/2310.02059>
- [5] Jingxuan He, Mark Vero, Gabriela Krasnopolka, and Martin Vechev. 2024. Instruction Tuning for Secure Code Generation. arXiv:2402.09497 [cs.CR] <https://arxiv.org/abs/2402.09497>
- [6] Nafis Tanveer Islam, Mohammad Bahrami Karkevandi, and Peyman Najafirad. 2024. Code Security Vulnerability Repair Using Reinforcement Learning with Large Language Models. arXiv:2401.07031 [cs.CR] <https://arxiv.org/abs/2401.07031>
- [7] Nafis Tanveer Islam, Joseph Khoury, Andrew Seong, Mohammad Bahrami Karkevandi, Gonzalo De La Torre Parra, Elias Bou-Harb, and Peyman Najafirad. 2024. LLM-Powered Code Vulnerability Repair with Reinforcement Learning and Semantic Reward. arXiv:2401.03374 [cs.SE] <https://arxiv.org/abs/2401.03374>
- [8] Jinghan Jia, Shashank Srikant, Tamara Mitrovska, Chuang Gan, Shiyu Chang, Sijia Liu, and Una-May O'Reilly. 2023. CLAWSAT: Towards Both Robust and Accurate Code Models. arXiv:2211.11711 [cs.LG] <https://arxiv.org/abs/2211.11711>
- [9] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure is Code Generated by ChatGPT? arXiv:2304.09655 [cs.CR] <https://arxiv.org/abs/2304.09655>
- [10] Junjie Li, Fazle Rabbi, Cheng Cheng, Aseem Sangal, Yuan Tian, and Jinqiu Yang. 2024. An Exploratory Study on Fine-Tuning Large Language Models for Secure Code Generation. arXiv:2408.09078 [cs.SE] <https://arxiv.org/abs/2408.09078>
- [11] Tomas Mikolov, Wen-tau Yih, and Geoffrey Zweig. 2013. Linguistic Regularities in Continuous Space Word Representations. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Lucy Vanderwende, Hal Daumé III, and Katrin Kirchhoff (Eds.). Association for Computational Linguistics, Atlanta, Georgia, 746–751. <https://aclanthology.org/N13-1090/>
- [12] Mahmoud Nazzal, Issa Khalil, Abdallah Khreishah, and NhatHai Phan. 2024. PromSec: Prompt Optimization for Secure Generation of Functional Source Code with Large Language Models (LLMs). In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) (CCS '24). Association for Computing Machinery, New York, NY, USA, 2266–2280. doi:10.1145/3658644.3690298
- [13] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 2205–2222. <https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval>
- [14] André Storhaug, Jingyue Li, and Tianyuan Hu. 2023. Efficient Avoidance of Vulnerabilities in Auto-completed Smart Contract Code Using Vulnerability-constrained Decoding. arXiv:2309.09826 [cs.CR] <https://arxiv.org/abs/2309.09826>
- [15] Rui Zhang, Yangfeng Ji, Yue Zhang, and Rebecca J. Passonneau. 2022. Contrastive Data and Learning for Natural Language Processing. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Tutorial Abstracts*, Miguel Ballesteros, Yulia Tsvetkov, and Cecilia O. Alm (Eds.). Association for Computational Linguistics, Seattle, United States, 39–47. doi:10.18653/v1/2022.naacl-tutorials.6